Data Science Lab: Project Report

Ali Ghorbanpour
Politecnico di Torino
Student id: s309992
s309992@studenti.polito.it

***Abstract*—In this report, a possible approach to the prediction of the number of shares associated with online news popularity is introduced. In particular, the model uses convolutional layers to automatically extract hierarchical patterns and features from input data. Then the proposed model will be evaluated with RMSE, which is a commonly used metric to measure the average magnitude of prediction errors in regression tasks.**

I. Problem Overview

Nowadays, the news is playing a crucial role in people's lives and their mindsets. So based on the degree of importance and validity of the news, people decide whether to rely on and pursue them or not. One approach to this problem is allocating a number of shares for the news. In order to predict online news popularity, which is the primary goal of this project, this study utilizes deep learning methods.

The dataset comprises both text and numerical values and has 39,797 records. It is divided into two parts:

- A development set with 31715 records that include shared numbers.
- An evaluation set with 7917 records that lack the shared numbers.

The distribution of the development dataset is represented in Figure 1. Also, this distribution showed specifically in Table I. One obvious thing is that we have an uneven distribution.

Separating the data into training and validation sets based on the validation loss values can help prevent overfitting during the training process. For example, when a tiny percentage (like 0.5 percent) of the distributed samples exceeds the value, e.g., 10,000, we have two approaches to separate the validation data:

- Before removing samples with shares values greater than 10,000, separate the validation data from the training data.
- After removing samples with shares values greater than 10,000, separate the validation data from the training data.

The first approach is suitable when the test data is randomly selected from the dataset, and the second one is suitable when the test data is randomly selected from the database after removing scattered samples or, by chance, there are no data with large share values in test data.

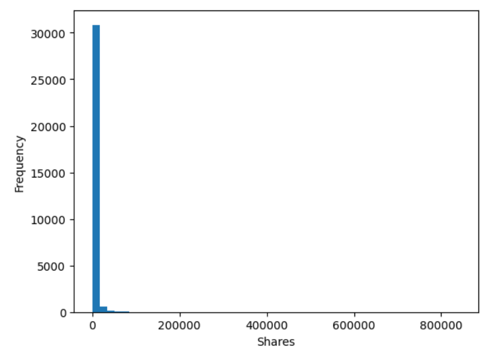

Fig. 1. Distribution of records based on frequency

Intervals	Number of samples
4.0 to 84333.6	31651
84333.6 to 168663.2	46
168663.2 to 252992.8	9
252992.8 to 337322.4	4
337322.4 to 421652.0	0
421652.0 to 505981.6	1
505981.6 to 590311.2	0
590311.2 to 674640.8	2
674640.8 to 758970.4	1
758970.4 to 843300.0	1

TABLE I
DISTRIBUTION OF THE DEVELOPMENT DATA

II. Proposed Approach

A. Preprocessing

First of all, we have to deal with NaN values. They can occur due to various reasons, such as data collection errors, incomplete records, or undefined calculations. For retaining the data, the NaN values present in certain features, such as *num_imgs* and *num_videos*, have to replace with a value that is out of the feature's range (e.g., -1). After that, the *weekday* and *data_channel* features containing string values are replaced with numerical values. Many machine learning algorithms and statistical models require numerical inputs. Numeric values are generally more computationally efficient to process than strings, allowing us to use a broader range of analysis techniques. This replacement is based on the average value of shares corresponding to the mentioned features. Figures 2 and 3 illustrate the replacement process for *weekday* and *data_channel*, respectively.

In the next step, the data distribution is examined based on the values of the shares. Figure 1 shows a highly uneven distribution, where samples with shares values exceeding 50,000 account for only 0.51 percent of the data. Considering the tremendous value of these samples and their limited number, removing them from the training data is very helpful because they introduce significant errors during training and disrupt the model's convergence. Figure 4 shows the data distribution after removing samples with a share value greater than 50,000. Next, the *URL* feature is removed. Neural networks can learn and adapt from data, which includes assigning varying levels of importance to different features or patterns in the data. It means that neural networks can learn to ignore or down-weight features that do not contribute significantly to the task.

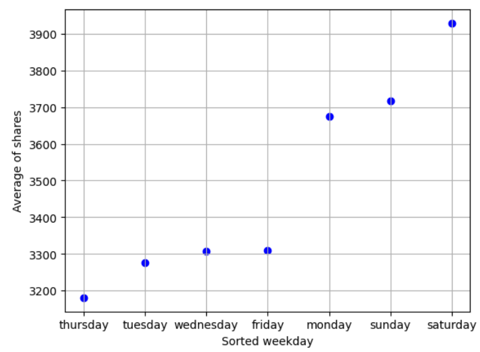

Fig. 2. Sorted *weekday*

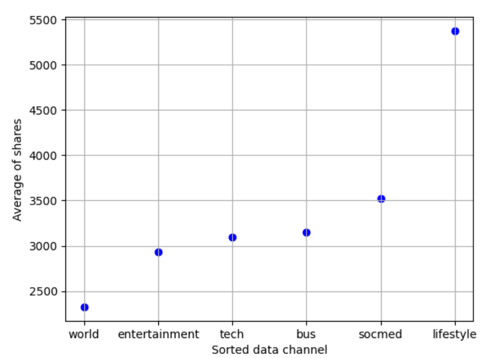

Fig. 3. Sorted *data_channel*

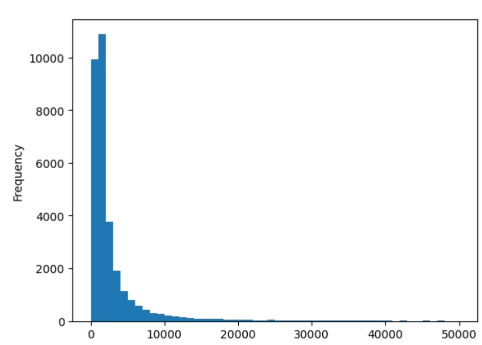

Fig. 4. Data distribution after removing records with a share value greater than 50,000

B. Model Selection

Once the data has been prepared, a model must be selected, tuned, and validated. Given the nature of the database, which consists of distinct features, each with only one value, a CNN (Convolutional Neural Network) model has been employed. CNN is known for its effectiveness in extracting complex features and performing tasks such as regression and classification. The used model is shown in Figure 5 where the model utilizes a neural network with five Conv1D layers and PReLU activation functions. This neural network has 10,874 trainable parameters.

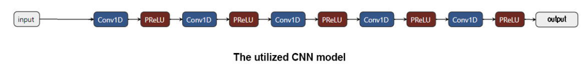

Fig. 5. Used CNN model

Conv1D is a convolutional neural network layer for processing one-dimensional sequential data. It applies a set of learnable filters to the input sequence, performing convolution operations to extract local patterns and features. The key idea behind Conv1D is to capture local dependencies and learn hierarchical representations in the sequential data. After that, an activation function is typically applied element-wise to introduce non-linearity into the network, enabling it to learn complex patterns

C. Hyperparameters tuning

This study employs a decreasing learning rate, where the initial value is set to 0.001 and multiplied by 0.99 every 200 steps. The use of a decreasing learning rate helps in achieving better model convergence. It is worth mentioning that each step involves passing a batch size (number of samples processed before the model is updated) number of samples through the neural network and performing back-propagation operations. The batch size for this study is set to 32. Also, the number of epochs is set to 100. An epoch represents a complete pass through the entire training dataset during the training phase of a model. The model takes the input data during an epoch and produces output predictions. These predictions are then compared to the actual targets or labels associated with the input data to calculate the loss.

III. Results

The best configuration for the CNN model was found for bach size = 32, epochs = 100, and learning_rate = 0.001 with RMSE = 5928.914. The results of training and evaluating the model are presented in Figure 6, which shows the RMSE values during the model training process. Finally, the distribution of the predicted values for evaluation samples is shown in Figure 7.

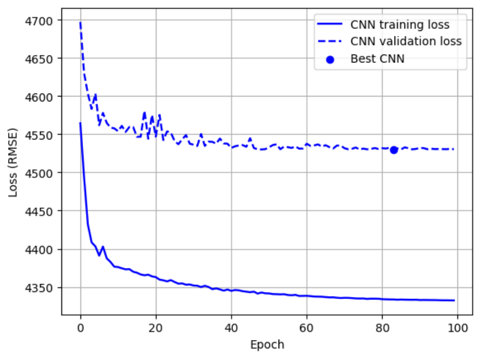

Fig. 6. Training and evaluating loss

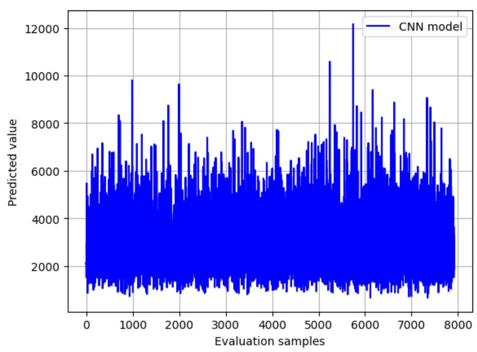

Fig. 7. Distribution of the predicted values

IV. Discussion

According to the adopted approach and the obtained results of the CNN model, it has been proven that this model is effective in handling non-linear relationships and categorical variables.

CNN (Convolutional Neural Network) is a type of deep learning model that refers to a subfield of machine learning that focuses on using Neural Networks with multiple layers to learn hierarchical representations of data. CNNs are specifically designed to leverage the spatial relationships present in the input data by using layers such as convolutional and pooling layers. These layers help capture local patterns and spatial hierarchies in the data, enabling the network to extract meaningful features automatically. By stacking multiple layers, CNNs can learn complex representations and make predictions based on the extracted features.

The following are some other models that might be worth considering to improve the obtained results further:

• Deep Neural Networks, also known as Multilayer Perceptrons (MLPs), are a versatile class of models that can handle non-linear relationships. DNNs consist of multiple hidden layers of neurons with non-linear activation functions. They can learn complex mappings between inputs and outputs and are widely used in various domains.

• Support Vector Machines (SVM) is a robust supervised learning algorithm that aims to find an optimal hyperplane in a high-dimensional feature space to separate data points of different classes. By utilizing kernel functions, SVMs can handle non-linear relationships by implicitly mapping the data to a higher-dimensional space. SVMs have been widely used in classification and regression tasks and have effectively handled various data types, including numerical and categorical variables.

• Techniques like Random Forest, AdaBoost, and Stacking can be used for regression tasks, combining the strengths of different models to achieve better results.

V. References

[1] ”TensorFlow” https://www.tensorflow.org/tutorials
[2] ”W3Schools” https://www.w3schools.com/python/